

Simulations numériques des problèmes
dépendant du temps

appliquées à l'épidémiologie, l'intelligence
artificielle et les finances

Gabriel Turinici

Chapitre 3

Différentiation automatique, backpropagation, optimisation, contrôle

3.1 Introduction

Le but de ce chapitre est de donner quelques intuitions sur comment faire d'une manière automatique le calcul du gradient (en disposant du code qui calcule la fonction). Il s'agit de la *différentiation automatique*; une façon de la mettre en oeuvre est la propagation rétrograde ('back-propagation' en anglais), et est utilisée dans des problèmes d'optimisation et de contrôle.

Donnons quelques exemples :

3.1.1 Exemple 1 : fonction explicite

Calcul du gradient d'une fonction à 3 variables $f(x, y, z) = (3x^2 + y)z - x$. Ce cas est assez simple, on peut faire à la main les opérations pour calculer $\nabla f = (\partial_x f, \partial_y f, \partial_z f)$. In nous servira de cas d'école et vérification.

3.1.2 Exemple 2 optimisation dans un modèle épidémiologique

Nous nous plaçons désormais dans un contexte actuel avec le système SIR, vu précédemment. Le système est de la forme $X' = f(X(t), u(t))$

$$\begin{cases} \dot{S} = -\beta SI \\ \dot{I} = \beta SI - \gamma I \\ \dot{R} = \gamma I \end{cases}$$

avec $u(t) = \beta(t) = \beta$ le contrôle.

Ainsi le but est de minimiser le coût du confinement ($\int_0^T c(\beta(t))dt$) et le nombre de gens atteints ($S(0) - S(T)$) au cours de la période $[0, T]$. Autrement dit, on cherche :

$$\min_{\beta} J[\beta] := S(0) - S(T) + \int_0^T c(\beta(t))dt$$

Nous avons dans un premier temps :

$$\frac{\partial}{\partial \beta} \left[\int_0^T c(\beta(t))dt \right] = c'(\beta(t)).$$

En revanche la différence $S(0) - S(T)$ est beaucoup moins évidente à dériver car le β n'apparaît pas explicitement, bien qu'il intervienne dans cette fonction.

3.1.3 Exemple 3 : réseaux neuronaux (NN)

Nous supposons que l'on possède une base de données Ω , on souhaite alors définir un réseau de neurones, *Neural Network* (NN), qui va "apprendre" par rapport à certains résultats de cette base de données, $\omega \in \Omega$.

On va alors chercher à optimiser certains paramètres X , pour que la sortie de notre réseau soit la plus proche des exemples ω choisis. C'est donc un problème d'optimisation :

$$\arg \min_X \{ \mathbb{E}_{\omega} (\mathcal{L}(X, \omega)) \}$$

On se propose d'étudier le réseau de neurones défini par le schéma suivant :

Le principe est alors comme suit : on dispose d'une couche d'entrée, appelée *Input Layer*, d'un certain nombre de couches cachées, les *Hidden Layers*, et une couche de sortie, l'*Output Layer*, voir figure 3.1 pour une illustration.

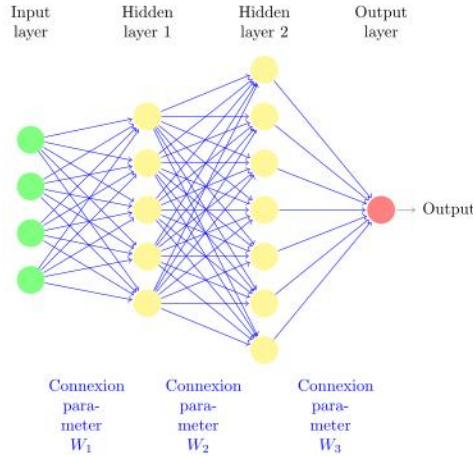


FIGURE 3.1 – Exemple d'un réseau de neurones

Chacune des couches cachées $(\tilde{Y}, Y)$ fait le calcul suivant : On prend tous les résultats de la couche précédente, on leur affecte un poids (un vecteur W , et un biais b). On obtient alors $\tilde{Y}$ et nous appliquons enfin une fonction d'activation $\mathcal{A}$ (par exemple la fonction ReLU qui est la partie positive : $ReLU(x_1, \dots, x_N) = ((x_1)_+, \dots, (x_N)_+)$, nous obtenons Y .

Enfin, nous appliquons à la dernière couche une fonction de sortie g (par exemple une fonction de type sigmoïde $x \mapsto \frac{1}{1+e^{-x}}$ qui transformera l'entrée en une sortie de type 'probabilité' c'est à dire un nombre entre 0 et 1). Si on doit choisir entre plusieurs classes, on peut utiliser comme sortie la fonction softmax $x \mapsto \left(\frac{e^{x_k}}{\sum_{\ell} e^{x_{\ell}}} \right)_k$ qui renvoie une loi de probabilité et en particulier la composante $\frac{e^{x_k}}{\sum_{\ell} e^{x_{\ell}}}$ du résultat peut s'interpréter comme étant la probabilité que l'étiquette

k soit la bonne.

D'une manière générale, on peut donc décrire la sortie Y_{n+1} d'une couche $n + 1$ donnée, par rapport à la sortie Y_n de la couche n précédente de la manière suivante :

$$\begin{cases} \tilde{Y}_{n+1} = W_{n+1}Y_n + b_{n+1} \\ Y_{n+1} = \mathcal{A}_{n+1}(\tilde{Y}_{n+1}) \end{cases}$$

On cherche alors à optimiser les poids W ainsi que les biais b , c'est-à-dire, à optimiser :

$$X = (W_1, b_1, \dots, W_n, b_n)$$

Pour minimiser une fonction de perte : $\mathcal{L}(X, \omega)$ qui décrit "l'écart entre l'estimation par le réseau de neurones avec les paramètres X , et les exemples ω ". Cette minimisation peut alors être effectuée avec une descente de gradient.

3.2 Approche par différences finies

Une première idée serait d'utiliser la formule

$$f'(x) = \frac{f(x+h) - f(x)}{h} + O(h). \quad (3.1)$$

Cette formule se démontre en utilisant la formule de Taylor. Elle porte le nom de formule de différences finies (non-centrées). Il y a d'ailleurs une autre formule plus précise

$$f'(x) = \frac{f(x+h) - f(x-h)}{2h} + O(h^2). \quad (3.2)$$

Elle se démontre également avec la formule de Taylor (exacte à l'ordre 2 avec reste d'ordre 3). L'approximation (3.2) porte le nom de différences finies centrées d'ordre 2.

Analysons maintenant le cout d'une telle formule pour une fonction générale $G(y_1, \dots, y_N)$. Il faut savoir qu'en général la partie la plus couteuse est l'évaluation de la fonction G .

Ainsi le cout sera exprimé en nombre d'évaluations de la fonction G nécessaires pour calculer $\nabla_X G$.

Par exemple si on veut obtenir $\partial_{y_k} G$ on peut utiliser l'approximation (3.2) pour la fonction $f(x) := G(y_1, \dots, y_{k-1}, x, y_k, \dots, y_N)$. Ceci veut dire que le calcul de $\partial_{y_k} G$ coutera deux évaluation de G . Donc le calcul de $\nabla_X G$ tout entier coutera $2N$ évaluations de G ce qui est un cout prohibitif lorsque N est grand (pour les réseaux neuronaux N peut être de l'ordre de 10^9 !). Il faut trouver une autre façon de faire.

En savoir plus 3.0.1. *Par contre les différence finies servent à vérifier de manière indépendante l'implémentation qui sera faite avec la différenciation automatique détaillée dans la suite.*



Mise en garde 3.0.1. *En général il faut aussi faire attention à la précision numérique. Si on suppose que la précision numérique du calcul de la fonction f est de 10^{-16} on aura donc une erreur de $\frac{10^{-16}}{2h} + h^2$; ceci peut être minimisé par rapport à h et obtenir l'ordre optimal d'erreur qui sera $O(10^{-10,66})$ (réalisé pour $h = O(10^{-16/3})$). Notons qu'en particulier on n'obtient jamais l'ordre 10^{-16} pour le calcul de la dérivée (ceci peut s'améliorer un peu en utilisant des formules d'ordre plus élevé que 2 ... mais qui ont un coût encore plus grand).*

3.3 Graphes computationnels, notions de *forward* et *backward*

3.3.1 Graphes computationnels directs

Nous abordons à présent le cœur du sujet : les graphes computationnels.

Définition 3.1. Graphe computationnel. Un graphe computationnel est un graphe orienté, connexe, sans cycles, dans lequel les nœuds correspondent à des opérations qui créent des nouvelles variables en utilisant les valeurs des variables des nœuds entrants.

Chaque variable peut donc être utilisée par un nœud cible, et chaque opération prend en source le résultat des opérations précédentes. Son propre résultat peut être alors utilisé par d'autres nœuds d'opérations cibles.

Prenons en exemple la fonction $f : \mathbb{R}^3 \rightarrow \mathbb{R}$ définie par :

$$f(x, y, z) = (3x^2 + y)z - x. \quad (3.3)$$

Nous pouvons associer à cette fonction f le graphe computationnel dans la figure 3.2.

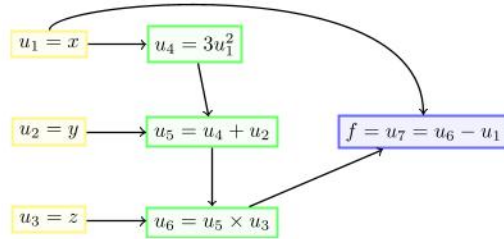


FIGURE 3.2 – Graphe computationnel de f dans l'équation (3.3).

Remarque 3.2. les nœuds jaunes (u_1, u_2, u_3) n'ont pas de degrés entrants, ce sont des nœuds qui ne contiennent que les variables d'entrée à notre fonction f . Le nœud bleu u_7 constitue la sortie de la fonction f . Ce graphe est dit direct : il effectue les mêmes opérations que celles de la fonction.

Définition 3.3 (Input, Output). On appelle Input d'un graphe computationnel direct, tous les nœuds ayant un degré

	U_4	U_5	U_6	U_7
nombre variables	1	2	2	2
fonction	$3U_1^2$	$U_4 + U_2$	$U_5 \times U_3$	$U_6 - U_1$

TABLE 3.1 – Fonctions des noeuds du graphe dans la figure 3.2.

entrant nul. On appelle *Output* d'un graphe computationnel tous les nœuds ayant un degré sortant nul.

Exemple 3.4. En reprenant la 3.2, les nœuds jaunes (u_1, u_2, u_3) sont des *Input*, le nœud bleu u_7 est un *output*.

Dans la table 3.1 on explicite les fonctions qui définissent chaque noeud du graphe dans la figure 3.2. Dans un problème d'optimisation, nous allons chercher à obtenir le gradient de la fonction calculée par la graphe. C'est à ce moment-là que la notion de graphe computationnel inverse intervient.

3.3.2 Graphes computationnels rétrogrades (backward)

Nous commençons par quelques rappels concernant le calcul des dérivées des fonctions composées. On suppose toutes les fonctions suffisamment régulières. Soit une fonction $f : \mathbb{R}^L \rightarrow \mathbb{R}$ et $g_1, \dots, g_L$ des fonctions de $\mathbb{R}^N$ dans $\mathbb{R}$ et on note $\forall X \in \mathbb{R}^N$:

$$F(X) := f(g_1(X), \dots, g_L(X))$$

La dérivée de la fonction F par rapport à X_k est donnée par :

$$\frac{\partial F}{\partial X_k} = \sum_{l=1}^L \frac{\partial f}{\partial g_l}(g_1(X), \dots, g_L(X)) \frac{\partial g_l}{\partial X_k}. \quad (3.4)$$

Nous rappelons que le gradient de la fonction F est le vecteur :

$$\nabla_X F = \left(\frac{\partial F}{\partial X_1}, \dots, \frac{\partial F}{\partial X_L} \right). \quad (3.5)$$

Parfois on notera sous forme "Jacobienne" $\nabla_X F$ comme $J_X F$.

En savoir plus 3.4.1. Nous rappelons la définition de la dérivée d'ordre 1 (Jacobienne) :

Définition 3.5 (Jacobienne). Soit $H : \mathbb{R}^N \rightarrow \mathbb{R}^P$ avec les notations, pour $X = (X_1, \dots, X_N) \in \mathbb{R}^N$:

$$H(X) = \begin{pmatrix} H_1(X) \\ \dots \\ H_P(X) \end{pmatrix}. \quad (3.6)$$

La matrice Jacobienne de H est la matrices des dérivées de H écrites comme suit

$$J_X H = \begin{pmatrix} \frac{\partial H_1}{\partial X_1} & \dots & \frac{\partial H_1}{\partial X_N} \\ \vdots & \dots & \vdots \\ \frac{\partial H_P}{\partial X_1} & \dots & \frac{\partial H_P}{\partial X_N} \end{pmatrix} \in \mathbb{R}^{P \times N}. \quad (3.7)$$

En particulier $(J_X H)_{ij} = \frac{\partial H_i}{\partial X_j}$. Parfois on utilise aussi la transposée $\frac{DH}{DX} = D_X H = (J_X H)^T$.

Remarque 3.6. La fonction $F : \mathbb{R}^N \rightarrow \mathbb{R}$ est en fait la fonction composée $f \circ g$. On a :

$$\left(\frac{\partial F}{\partial X_1}, \dots, \frac{\partial F}{\partial X_L} \right) = J_X F = J_X (f \circ g) = J_g f \times J_X g, \quad (3.8)$$

où le produit " $\times$ " est le produit matrice-vecteur usuel. C'est une formule de dérivation de fonctions composées.

Revenons au graphe 3.2; nous employons la notation suivante :

$$\delta U_k = \frac{\partial f}{\partial U_k}. \quad (3.9)$$

Évidemment, $\delta U_7 = \delta f = 1$. Pour les autres ce n'est pas si immédiat mais on voit de suite que ce calcul semble être plus facile à développer à l'inverse, c'est pourquoi il sera dit

rétrograde. Soyons plus précis.

Pour utiliser la formule (3.4) il faut savoir mettre le calcul sous cette forme. Ce n'est pas possible avec le graphe initial mais avec une version 'par couche' comme dans la figure 3.3, ou chaque *couche* contient toutes les variables utiles pour calculer les valeurs de la couche suivante : la dernière couche contient seulement U_7 , sa dérivée est déjà calculée. La couche avant-dernière contient seulement U_6 et $U_{1,4}$ (qui propage la valeur de x). On peut utiliser la dérivation composée pour obtenir $\delta U_6 = \frac{\partial U_7}{\partial U_6} = 1$, d'après la formule de U_7 dans le tableau 3.1 ; de la même façon $\delta U_{1,4} = \frac{\partial U_7}{\partial U_{1,4}} = -1$.

On continue donc en arrière ; cette nouvelle couche contient $U_{1,3}$, U_5 et $U_{3,3}$. L'idée est de voir $U_{1,3}$, U_5 et $U_{3,3}$, comme des entrées, la couche suivante U_6 et $U_{1,4}$ comme des variables intermédiaires et U_7 comme la sortie. On applique alors la formule de dérivation composée, par exemple :

$$\delta U_5 = \frac{\partial f}{\partial U_5} = \frac{\partial f}{\partial U_6} \cdot \frac{\partial U_6}{\partial U_5} + \frac{\partial f}{\partial U_{1,4}} \cdot \frac{\partial U_{1,4}}{\partial U_5} = \delta U_6 \cdot U_{3,3} + 0 = U_{3,3}.$$

Le calcul peut ainsi continuer. Attention au calcul final de δU_1 qui aura deux termes non nuls.

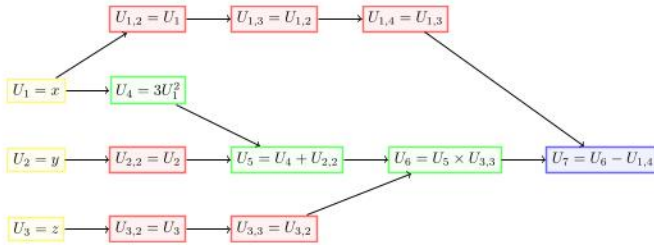


FIGURE 3.3 – Graphe computationnel par couche de f dans l'équation (3.3). Il a été construit en ajoutant des variables qui seront nécessaires plus tard sous la forme de nouvelles variables triviales, comme $U_{1,2} = U_1$, etc.

Remarque 3.7. Cette représentation 'par couche' introduit des variables supplémentaires, ici en rouge, mais a

l'avantage d'avoir une structure par bloc où chaque bloc utilise seulement le bloc qui le précède strictement. Cette solution a l'avantage d'être rigoureuse mais l'inconvénient de compliquer les calculs.

Pour simplifier les calculs l'idée est de garder le graphe initial, mais de respecter la règle suivante : en allant en arrière de la sortie vers les entrées, on calcule la dérivée correspondante à une variable U_a seulement lorsque toutes les dérivées δU_b des cases U_b tel qu'une flèche entre U_a et U_b existe ont été calculées. Plus simplement dit : lorsque tout le monde qui suit U_a dans le graphe a son dérivée calculée alors on peut aussi calculer la dérivée δU_a . Dans notre cas de figure ceci suppose par exemple tenter le calcul de δU_1 seulement une fois que δU_4 et $\delta U_{1,2}$ sont connues. On obtient le calcul dans la figure 3.4.

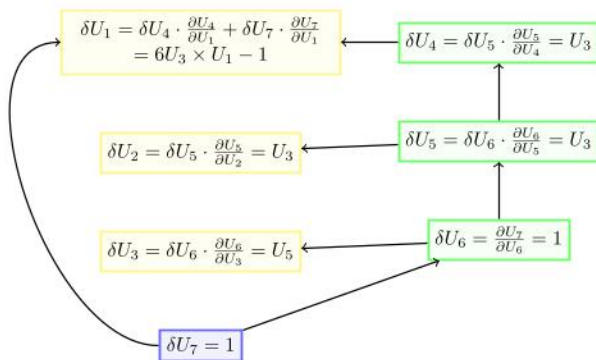


FIGURE 3.4 – Graphe rétrograde de f en équation (3.3).

Pour résumer, il est possible de créer un graphe computationnel **direct** (qui effectue les mêmes opérations que notre fonction) ou bien un graphe computationnel **rétrograde** (qui contient les dérivées).

3.4 Exemple 1 : Réseaux neuronaux

Nous supposons que l'on possède une base de données dite d'apprentissage Ω , on souhaite alors définir un réseau de neurones, *Neural Network* (NN), qui va "apprendre" par rapport à cette base de données, $\omega \in \Omega$.

On va alors chercher à optimiser certains paramètres X , pour que la sortie de notre réseau soit la plus proche possible de la sortie (connue) pour les exemples ω de la base Ω (pour tout $\omega \in \Omega$ on connaît la sortie correcte; on est ainsi dans le cadre de l'apprentissage dit *supervisé*). C'est donc un problème d'optimisation :

$$\arg \min_X \{ \mathbb{E}_\omega (\mathcal{L}(X, \omega)) \} \quad (3.10)$$

On se propose d'étudier le réseau de neurones défini par le schéma illustré dans la figure [3.1](#).

3.4.1 Définition du problème

Le réseau est constitué d'une couche d'entrée, appelée *Input Layer*, d'un certain nombre de couches cachées, les *Hidden Layers*, et une couche de sortie, l'*Output Layer*. Chacune des couches cachées $(\tilde{Y}, Y)$ fait le calcul suivant : on prend tous les résultats de la couche précédente, on leur affecte un poids (un vecteur W , et un biais b). On obtient alors $\tilde{Y}$ et nous appliquons enfin une fonction d'activation $\mathcal{A}$, pour obtenir Y . Enfin, nous appliquons à la dernière couche une fonction de sortie g . D'une manière générale, on peut donc décrire la sortie Y_{n+1} d'une couche $n+1$ donnée, par rapport à la sortie Y_n de la couche n précédente de la manière suivante :

$$\begin{cases} \tilde{Y}_{n+1} = W_{n+1}Y_n + b_{n+1} \\ Y_{n+1} = \mathcal{A}_{n+1}(\tilde{Y}_{n+1}) \end{cases}$$

On cherche alors à optimiser les poids W ainsi que les biais b , c'est-à-dire, à optimiser :

$$X = (W_1, b_1, \dots, W_n, b_n)$$

Pour minimiser une fonction de perte : $\mathcal{L}(X, \omega)$ qui décrit "l'écart entre l'estimation par le réseau de neurones avec les paramètres X , pour les exemples ω ". Cette minimisation peut alors être effectuée par exemple avec un algorithme de descente de gradient stochastique *Stochastic Gradient Descent* (*SGD*).

3.4.2 Graphe computationnel du réseau de neurones

Dans notre exemple défini par le graphe dans la figure 3.1, nous avons :

- Une *Input Layer*, $Y_0 \in \mathbb{R}^4$;
- Un premier *Hidden Layer*, $(\tilde{Y}_1, Y_1) \in \mathbb{R}^5$;
- Un second *Hidden Layer*, $(\tilde{Y}_2, Y_2) \in \mathbb{R}^7$;
- Une *Output Layer*, $O = g(Y_2) \in \mathbb{R}$.

Nous disposons donc du graphe computationnel présenté dans la figure 3.5.

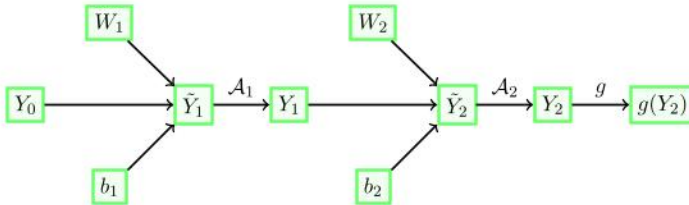


FIGURE 3.5 – Graphe de propagation forward du réseau de neurones

3.4.3 Calcul du gradient de la fonction de perte

Nous allons désormais nous intéresser au graphe rétrograde de ce réseau de neurones ainsi qu'au calcul du gradient :

Nous avons alors (attention c'est une écriture symbolique, pour la rendre rigoureuse il faut employer la matrice Jaco-

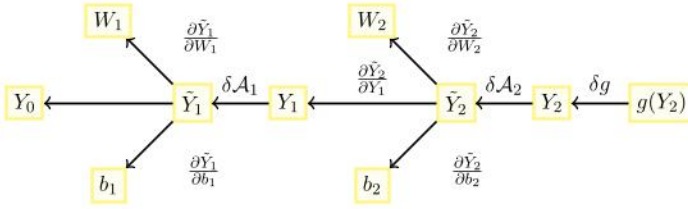


FIGURE 3.6 – Graphe de propagation backward du réseau de neurones

bienn, voir l'encadré 3.4.1 page 56) :

$$\left\{ \begin{array}{l} \frac{\partial g}{\partial W_2} = \frac{\partial g}{\partial Y_2} \cdot \frac{\partial Y_2}{\partial W_2} = \frac{\partial g}{\partial Y_2} \cdot \frac{\partial Y_2}{\partial Y_2} \cdot \frac{\partial \tilde{Y}_2}{\partial W_2} \\ \frac{\partial g}{\partial b_2} = \frac{\partial g}{\partial Y_2} \cdot \frac{\partial Y_2}{\partial \tilde{Y}_2} \cdot \frac{\partial \tilde{Y}_2}{\partial b_2} \\ \frac{\partial g}{\partial W_1} = \frac{\partial g}{\partial Y_2} \cdot \frac{\partial Y_2}{\partial W_1} = \frac{\partial g}{\partial Y_2} \cdot \frac{\partial Y_2}{\partial \tilde{Y}_2} \cdot \frac{\partial \tilde{Y}_2}{\partial W_1} \\ = \frac{\partial g}{\partial Y_2} \cdot \frac{\partial Y_2}{\partial \tilde{Y}_2} \cdot \frac{\partial \tilde{Y}_2}{\partial Y_1} \cdot \frac{\partial Y_1}{\partial W_1} = \frac{\partial g}{\partial Y_2} \cdot \frac{\partial Y_2}{\partial \tilde{Y}_2} \cdot \frac{\partial \tilde{Y}_2}{\partial Y_1} \cdot \frac{\partial Y_1}{\partial \tilde{Y}_1} \cdot \frac{\partial \tilde{Y}_1}{\partial W_1} \\ \frac{\partial g}{\partial b_1} = \frac{\partial g}{\partial Y_2} \cdot \frac{\partial Y_2}{\partial \tilde{Y}_2} \cdot \frac{\partial \tilde{Y}_2}{\partial Y_1} \cdot \frac{\partial Y_1}{\partial \tilde{Y}_1} \cdot \frac{\partial \tilde{Y}_1}{\partial b_1} \end{array} \right.$$

Il est donc possible de calculer le quadruplet : $\left[\frac{\partial g}{\partial W_1}, \frac{\partial g}{\partial b_1}, \frac{\partial g}{\partial W_2}, \frac{\partial g}{\partial b_2} \right]$ qui nous donne le gradient de g par rapport à $X = (W_1, b_1, W_2, b_2)$. Il suffit alors d'optimiser g par rapport à X avec une descente de gradient.

Remarque 3.8. Notons qu'en réalité nous avons besoin surtout du dérivé de la fonction "loss" $\mathcal{L}$ dans (3.10) ; elle en découle directement de la dérivée de g . Par ailleurs, la construction de la fonction de loss tient compte aussi bien des objectifs (reproduction des résultats connus) mais également de autres considérations, par exemple pouvoir de généralisation ; en effet, il faut empêcher le sur-apprentissage, (overfit). Pour y remédier, nous pouvons instaurer une pénalité dans la correction apportée aux poids à chaque itérations de la descente de gradient, ou bien effectuer un dropout, c'est à dire supprimer de temps

un temps un neurone pour forcer l'algorithme à ne pas sur-apprendre.

Remarque 3.9. Le gradient exprimé ci-dessus correspond à la proposition montrée plus tôt : chacune des dérivées partielles peut être exprimée comme une somme.

En savoir plus 3.9.1 (Descente de gradient stochastique). Nous rappelons la formule de la descente de gradient pour une fonction multidimensionnelle $\mathcal{F} : \mathbb{R}^p \rightarrow \mathbb{R}$, de gradient $\nabla_x \mathcal{F}$:

$$x_{n+1} = x_n - \gamma_n \nabla_x \mathcal{F}(x_n).$$

Sous certaines hypothèse (convexité, taille de γ , ...) la suite $(x_n)_{n \geq 1}$ converge vers le minimum local le plus proche.

Ici γ_n est aussi appelée "taux d'apprentissage" dans le cas d'apprentissage statistique. Dans le cas de grandes dimensions et lorsque la fonction $\mathcal{F}$ a la forme d'une moyenne $\mathcal{F} = \mathbb{E}_\omega F(x, \omega)$ comme en équation (3.10), le calcul de gradient en un point devient très couteux (et donc obtenir la moyenne nécessite une moyenne empirique couteuse à calculer) ; c'est la descente de gradient stochastique (SGD) qui est utilisée. Nous prendrons alors, parmi les différents gradients $\nabla_x F(x, \omega)$ une valeur (ou plusieurs, mais en nombre petit) tiré aléatoirement.

$$x_{n+1} = x_n - \gamma_n \nabla_x F(x_n, \omega_n).$$

Plusieurs algorithmes de descente de gradient stochastique couplent différentes techniques (de variation de pas d'apprentissage ainsi que de descente stochastique).

3.5 Exemple 2 : problème de contrôle

Nous nous plaçons désormais dans le contexte du système SIR, vu précédemment. Par contre nous allons commencer par une présentation plus générale en considérant qu'on utilise la méthode d'Euler Explicite pour résoudre $x' = f(t, x(t), u(t))$ avec la fonction $F(x(T))$ à optimiser.

3.5.1 Graphes computationnels

La première manière de résoudre ce problème est de faire les graphes computationnels. Dans un premier temps, nous obtenons le graphe computationnel direct dans la figure 3.7 haut. Nous obtenons par la suite le graphe computationnel rétrograde dans la figure 3.7 bas.

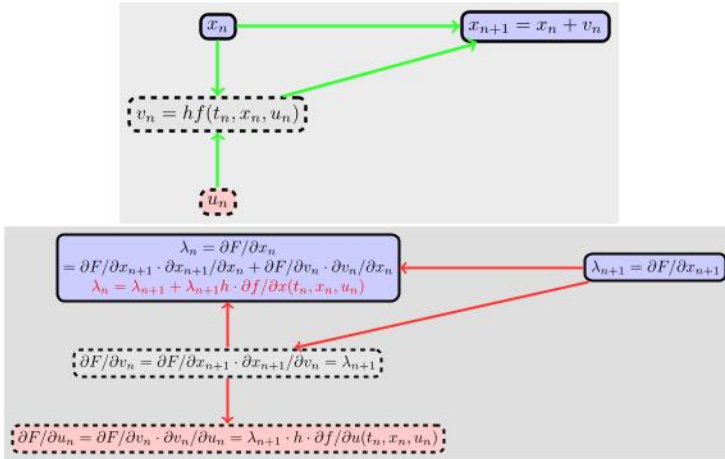


FIGURE 3.7 – Description du mode "forward" (image de haut) et "backward" (image de bas) pour un problème de contrôle; le schéma obtenu pour l'état adjoint λ_n est une discrétisation de type Euler Explicite de (3.13). Bien que ceci semble correspondre à Euler Implicite, il faut se rappeler que (3.13) se résout à l'inverse dans le temps, ce qui veut dire que par exemple λ_{n+1} est connu avant λ_n .

Cependant, ces graphes ne représentent qu'une partie du graphe réel. En effet, le graphe direct complet est, pour $n = 2$, présenté en figure 3.8.

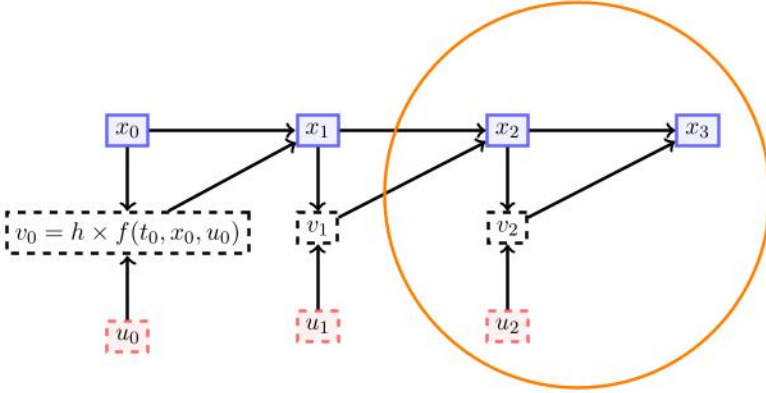


FIGURE 3.8 – Graphe direct complet du système SIR pour $n = 2$; le cercle orange représente ici la partie que nous avons tracée précédemment en 3.7.

3.5.2 Construction d'une version discrète d'une procédure d'Euler Lagrange

Les graphes computationnels directs et rétrogrades nous permettent de construire une version discrète d'une procédure d'Euler Lagrange.

Considérons un cadre général où $x(t)$ satisfait l'EDO : $x'(t) = f(t, x(t), u(t))$. Au niveau discret le schéma EE donne $x_{n+1} = x_n + hf(t_n, x_n, u_n)$. Le graphe computationnel nous permet d'obtenir :

$$\lambda_n = \lambda_{n+1} + \lambda_{n+1} \frac{\partial f}{\partial X}(t_n, x_n, u_n)h \quad (3.11)$$

$$\lambda_N = \frac{\partial F}{\partial x_N}. \quad (3.12)$$

La particularité évidente de cette formule est qu'on connaît la fin, à savoir λ_N et on doit construire les autres valeurs λ_n .

Ceci nous conduit à voir $\lambda(t)$ comme solution d'une équation **rétrograde dans le temps** et donc on constate qu'on est en train de résoudre, par une méthode d'Euler Explicite, l'équation :

$$\lambda'(t) = -\frac{\partial f}{\partial x}(t, x(t), u(t))\lambda(t). \quad (3.13)$$

Nous obtenons également une formule pour la dérivée :

$$\frac{\partial F}{\partial u_n} = \lambda_{n+1} h \frac{\partial f}{\partial u_n}(t_n, x_n, u_n). \quad (3.14)$$

Pour résumer, le graphe computationnel nous montre que la dérivée peut être obtenue avec l'aide d'une résolution de l'EDO (3.13) faisant intervenir $\lambda(t)$ qui sera appelé état adjoint. Il est important de voir que le coût de cette méthode de calcul est juste 2 fois le coût du calcul de F , et donc indépendant du nombre N de valeurs u_n à optimiser (à comparer avec $2N$ évaluations qui seraient nécessaires si utilisation d'une formule de différences finies comme (3.2)).

En savoir plus 3.9.2. La relation (3.14) est cohérente avec une formulation du problème comme une minimisation de type Euler Lagrange ; il ne s'agit par d'une démonstration, plutôt d'une vérification. La minimisation à faire s'écrit comme $\min_{x'=f(t,x,u)} F(x(T))$. Nous appliquons à la fonction $F(x(t))$ la méthode du multiplicateur de Lagrange, et nous introduisons G :

$$G(x, u, \lambda) = F(x(T)) - \int_0^T (x' - f(t, x, u))\lambda(t)dt,$$

où $\lambda(t)$ est le multiplicateur de Lagrange. Il convient ensuite d'annuler les dérivées partielles de G en fonction de λ et de x . Comme attendu, la dérivée par rapport au multiplicateur λ nous permet d'obtenir la

contrainte :

$$\frac{\partial G}{\partial \lambda(t)} = 0 \Leftrightarrow x' = f(t, x, u).$$

La dérivée par rapport au multiplicateur de Lagrange $\lambda(t)$ va nous permettre de faire le lien entre notre graphe computationnel et la procédure d'Euler Lagrange :

$$\begin{aligned} \frac{\partial G}{\partial x(t)} &= -\frac{\partial}{\partial x(t)} \int_0^T \lambda(x' - f(t, x, u)) dt \\ &\stackrel{IPP}{=} -\frac{\partial}{\partial x(t)} \left[[\lambda x]_0^T - \int_0^T x \lambda' - \lambda f dt \right] \\ &= \lambda' - \lambda \frac{\partial f}{\partial x}(t, x(t), u(t)). \end{aligned}$$

Nous retrouvons l'équation (3.13). Donc le graphe computationnel nous permet d'obtenir une version discrète de la procédure d'Euler Lagrange pour notre variable de contrôle. La dérivée par rapport à $x(T)$ permet de trouver que $\lambda(T)$ doit être pris égal à $\frac{\partial F(x(T))}{\partial x(T)}$.

3.5.3 Rappel du problème

Appliquons maintenant la procédure décrite pour notre situation du modèle SIR. Le problème peut être représentés par un système, que l'on intitule SIR. Le système est de la forme $x' = f(t, x(t), u(t))$

$$\begin{cases} \dot{S} = -\beta SI \\ \dot{I} = \beta SI - \gamma I \\ \dot{R} = \gamma I \end{cases}$$

avec $u(t) = \beta(t) = \beta$ le contrôle et $x = (S, I, R)$ l'état. Le but est de minimiser le coût du confinement $(\int_0^T c(\beta(t))dt)$ et le

nombre de gens atteints ($S(0) - S(T)$) au cours de la période $[0, T]$. Autrement dit, on cherche :

$$\min_{\beta} S(0) - S(T) + \int_0^T c(\beta(t))dt$$

Nous noterons par la suite $J(\beta) = S(0) - S(T) + \int_0^T c(\beta(t))dt$, avec u , l'ensemble des fonctions u_n du temps que l'on va discrétiser :

$$\begin{array}{ccccccc} & u_1 & & u_2 & & u_3 & \\ | & & | & & | & & \\ 0 & h & 2h & 3h & & & T = N \cdot h \end{array}$$

Nous avons dans un premier temps :

$$\frac{\partial}{\partial \beta} \left[\int_0^T c(\beta(t))dt \right] = c'(\beta(t)).$$

En revanche la différence $S(0) - S(T)$ est beaucoup moins évidente à dériver car le β n'apparaît pas explicitement, bien qu'il intervienne dans cette fonction. D'une manière générale, nous cherchons donc à calculer : $\frac{\partial}{\partial u_n} [F(x(T))]$, (pour SIR $F(x(T)) = S(0) - S(T)$).

La méthodologie présentée en [3.9.2](#) peut s'appliquer et on obtient que, si on introduit, λ et μ qui vérifient :

$$\lambda'(t) = \beta(t)I(t)(\lambda(t) - \mu(t)), \quad \lambda(T) = -1, \quad (3.15)$$

$$\mu'(t) = \beta(t)S(t)(\lambda(t) - \mu(t)) + \gamma\mu(t), \quad \mu(T) = 0. \quad (3.16)$$

alors $S(T) = \int_0^T -\beta(t)S(t)I(t)(\lambda(t) - \mu(t))dt - S(0)\lambda(0) - I(0)\mu(0)$, qui peut être maintenant dérivé par rapport à $\beta(t)$ pour obtenir $\frac{\partial}{\partial \beta(t)} S(T) = -S(t)I(t)(\lambda(t) - \mu(t))$.

3.6 Annexe théorique : théorie des graphes

Nous allons introduire quelques notions de théorie des graphes.

Définition 3.10 (Graphe orienté). Un graphe orienté est un couple $G = (V, A)$ comprenant :

- V un ensemble de nœuds ;
- A un ensemble d'arcs, $A \subseteq V^2$ chaque arc étant associé à deux nœuds, avec une direction.

Exemple 3.11. Voir figure [3.9](#) pour un exemple de graphe orienté.

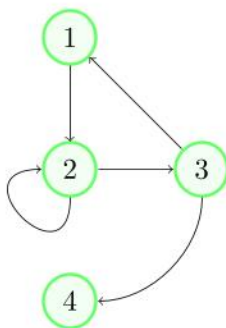


FIGURE 3.9 – Exemple de graphe orienté

Chaque arc $\{x, y\}$ va donc du nœud x au nœud y . L'arc $\{x, y\}$ n'est donc pas le même que l'arc $\{y, x\}$. Il est possible qu'un arc pointe vers le nœud duquel il part ($\{x, x\}$).

Remarque 3.12. Un graphe non orienté est un couple $G = (V, E)$ où $E \subseteq \{(x, y), (y, x) \mid x, y \in V, x \neq y\}$ est un ensemble d'arêtes qui relient deux nœuds différents entre eux. Un arc est donc une arête avec une direction. On parle de chaîne d'un nœud à un autre pour parler d'une suite consécutive d'arêtes qui relient les deux nœuds.

Définition 3.13 (Graphe connexe). Un graphe est dit connexe s'il est d'un seul tenant, c'est-à-dire si pour toute paire de nœuds $v_1, v_2 \in V$ il existe une chaîne allant du nœud v_1 au nœud v_2 .

Exemple 3.14. Voir figures [3.10](#) et [3.11](#) pour un exemple de graphe connexe ou pas connexe.

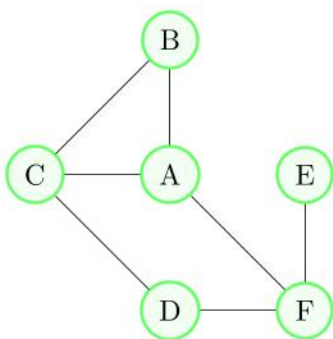


FIGURE 3.10
— Graphe
connexe

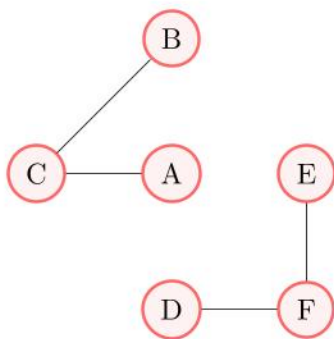


FIGURE 3.11
— Graphe
non
connexe

Définition 3.15. Graphe sans cycles. *Un graphe sans cycles est un graphe qui ne contient aucune chaîne qui possède le même nœud de départ et d'arrivée.*

Le graphe de la Figure 3.10 possède plusieurs cycles (notamment $\{A, F, D, C\}$), celui de la Figure 3.11 n'en possède aucun.

Définition 3.16 (Degrés d'un nœud). *Etant donné un arc $\{x, y\}$ reliant les nœuds x et y , on appelle le nœud de départ x la **source** et le nœud d'arrivée y la **cible**. On définit alors :*

- le **degré sortant** d'un nœud est le nombre d'arcs qui ont pour source ce nœud ;
- le **degré entrant** d'un nœud est le nombre d'arcs qui ont pour cible ce nœud.

Un graphe comporte donc autant de degrés entrants que de degrés sortants.

Exemple 3.17. *Dans le graphe de la 3.9 :*

- le degré sortant du nœud $\{2\}$ est 2, le degré sortant du nœud $\{4\}$ est nul.
- le degré entrant du nœud $\{3\}$ est 1, le degré entrant du nœud $\{2\}$ est 2.

- *Il y a au total 5 degrés entrants et 5 degrés sortants, ce qui corrobore notre définition : il y a autant de degrés entrants que de degrés sortants.*

3.7 Exercices

Exercice 3.1 (différences finies d'ordre 3 et 4).

1. Montrer que

$$f'(x) = \frac{2f(x+3h) - 9f(x+2h) + 18f(x+h) - 11f(x)}{6h} + O(h^3). \quad (3.17)$$

2. Trouver une formule d'ordre 3 du type :

$$f'(x) = \frac{\alpha f(x+2h) + \beta f(x+h) + \gamma f(x) + \delta f(x-h)}{h} + O(h^3). \quad (3.18)$$

3. Montrer que

$$f'(x) = \frac{-f(x+2h) + 8f(x+h) - 8f(x-h) + f(x-2h)}{12h} + O(h^4). \quad (3.19)$$

Exercice 3.2 (Rappel multiplicateurs Euler-Lagrange -1-).

Trouver le minimum de la fonction $x + y$ sous la contrainte $x^2 + y^2 = 1$ par la méthode des multiplicateurs d'Euler Lagrange.

Exercice 3.3 (Rappel multiplicateurs Euler-Lagrange -2-).

Soit $f, g : \mathbb{R}^2 \rightarrow \mathbb{R}$ de classe C^2 . Supposons que l'équation $g(x, y) = 0$ admet pour chaque x donné, une solution unique $y = \mathcal{Y}(x)$ avec $\mathcal{Y}$ de classe C^1 . On suppose aussi que pour tout $x : \nabla_y g(x, \mathcal{Y}(x)) \neq 0$. Notons $F : \mathbb{R} \rightarrow \mathbb{R}$ où $F(x) = f(x, \mathcal{Y}(x))$. On suppose $\mathcal{Y}(\cdot)$ difficile à obtenir et on veut calculer le gradient $\nabla_x F(x)$ sans utiliser trop de fois $\mathcal{Y}(\cdot)$. Soit alors $L(\lambda, x, y) = f(x, y) + \lambda g(x, y)$.

1. Montrer que pour tout x il existe λ_x tel que :

$$\nabla_y L(\lambda_x, x, \mathcal{Y}(x)) = \nabla_\lambda L(\lambda_x, x, \mathcal{Y}(x)) = 0. \quad (3.20)$$

2. Montrer que : $\nabla_x F(x) = \nabla_x L(\lambda_x, x, \mathcal{Y}(x))$.

3. Généraliser pour des fonctions $f : \mathbb{R}^n \times \mathbb{R}^n \rightarrow \mathbb{R}$,
 $g : \mathbb{R}^n \times \mathbb{R}^n \rightarrow \mathbb{R}^n$.

Exercice 3.4 (exemple de graphe computationnel).

1. Écrire le graphe computationnel du calcul suivant :
 $z = x + \sin(x \cdot y) + x^2$. Écrire ensuite le graphe backward pour le calcul des dérivées de z par rapport à x et y . Calculer explicitement les valeurs de ces deux graphes pour $x = 2$, $y = \pi/6$.
2. Pareil pour la fonction $(x, y, z) \mapsto xyz e^{x^2+yz}$.

Exercice 3.5 (Mode backward EI, cf figure 3.7).

1. Trouver les formules pour le mode forward et backward pour un problème de contrôle comme dans la figure 3.7 mais avec un schéma de Euler Implicite.
2. Pareil si le graphe forward implémente un schéma RK général, trouver le schéma "adjoint" c'est à dire celui que sera employé par la propagation rétrograde de l'état adjoint. Qu'observez vous ?

Exercice 3.6 (graphe computationnel de la projection). Soit $P = \{(x, y, z)^T \in \mathbb{R}^3 \text{ tels que } x + y + z = 1\} \subset \mathbb{R}^3$ et $\Pi : \mathbb{R}^3 \rightarrow P$ définie pour tout vecteur $v = (v_1, v_2, v_3)^T \in \mathbb{R}^3$ par $\Pi(v) = v - \lambda(v) \cdot (1, 1, 1)^T$ avec $\lambda(v) \in \mathbb{R}$ le seul nombre réel tel que $\Pi(v) \in P$. Soit $g : \mathbb{R}^3 \rightarrow \mathbb{R}$ une fonction de classe C^1 .

1. Trouver la formule de λ_v en fonction de v ;
2. Écrire le graphe computationnel direct qui calcule $v \mapsto g(\Pi(v))$;
3. Écrire le graphe computationnel rétrograde qui calcule les dérivées de $g(\Pi(v))$ par rapport aux entrées (composantes de v).
4. Faire le calcul des dérivées pour $v = (1, 2, 3)^T$ et $g(x, y, z) = x^2 + yz - 2$.

Rappel : pour tout vecteur a nous notons par a^T son transposé.

Exercice 3.7 (graphe computationnel softmax/cross entropy). Calculer la fonction g pour le cas de la $\text{loss}=\text{cross-entropy}$ après une dernière couche "softmax" (output dans $\mathbb{R}^3$) et architecture FC/ReLU 4 – 5 – 7 – 3 ; calculer ses dérivées.

Exercice 3.8 ("Layer Normalization"). Une partie du travail de la couche dite "Layer Normalization" est le calcul de l'écart type d'un échantillon de données $x = (x_1, \dots, x_n)$. Il se fait de la manière suivante : tout d'abord calcul de la moyenne empirique $\bar{\mu} = (\sum_{k=1}^n x_k)/n$, ensuite de la variance empirique $v = \sum_{k=1}^n (x_k - \bar{\mu})^2 / (n - 1)$ et finalement de l'estimation de l'écart type $\bar{\sigma} = \sqrt{v}$.

1. dessiner le graphe computationnel direct correspondant aux calculs ci-dessus ; pour chaque nœud du graphe préciser explicitement, comme dans le cours, les input / outputs et l'opération faite par le nœud.
2. pareil pour le graphe computationnel rétrograde des dérivées de la sortie finale par rapport aux entrées ;
3. Faire le calcul des dérivées pour $n = 3$, $x = (1, 2, 4)$ avec l'aide du graphe computationnel direct et celui rétrograde.

3.8 TP chapitre backward

Exercice 3.9. Implémenter l'exo [3.5](#) pour le cas du modèle SIR en section [3.1.2](#). Prendre les valeurs $T = 150, N = 150, h = T/N$ $S(0) = 10^6, I(0) = 10, R(0) = 0, N_{\text{total}} = S(0) + I(0) + R(0)$ $\beta = 0.5/N_{\text{total}}, \gamma = 1/3, c(\beta) = c_0/\beta$ avec $c_0 = 10^{-2}$.

Exercice 3.10. Construire et entrainer un réseau neuronal à couches denses décrit en section [3.1.3](#).